

# 第4章 软件架构的风格与模式

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

# 第4章 软件架构的风格与模式

- 4.0 引言
- 4.1 软件架构风格定义
- 4.2 软件架构风格的分类
- 4.3 典型的软件架构风格
- 4.4 软件架构模式

## 4.2 软件架构风格的分类

- (1) 数据流风格：批处理、管道/过滤器；
- (2) 调用/返回风格：主程序/子程序、面向对象风格、层次结构；
- (3) 以数据为中心的风格：仓库风格、超文本系统、黑板风格等；
- (4) 虚拟机风格：解释器、基于规则的系统；
- (5) 独立组件风格：进程通讯、事件系统。

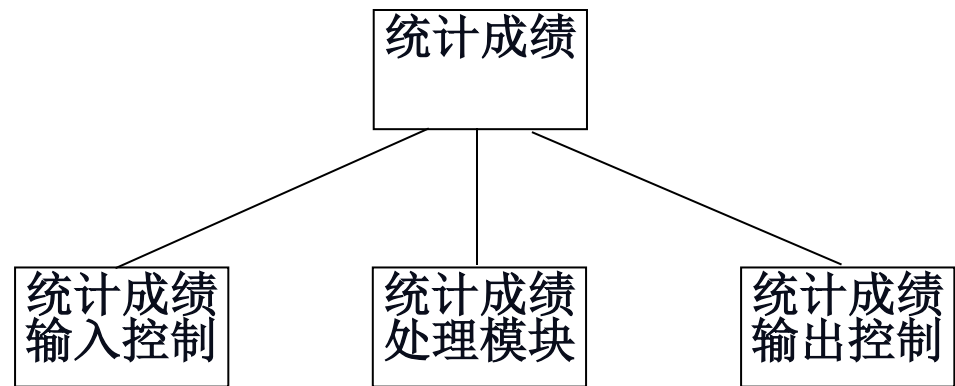
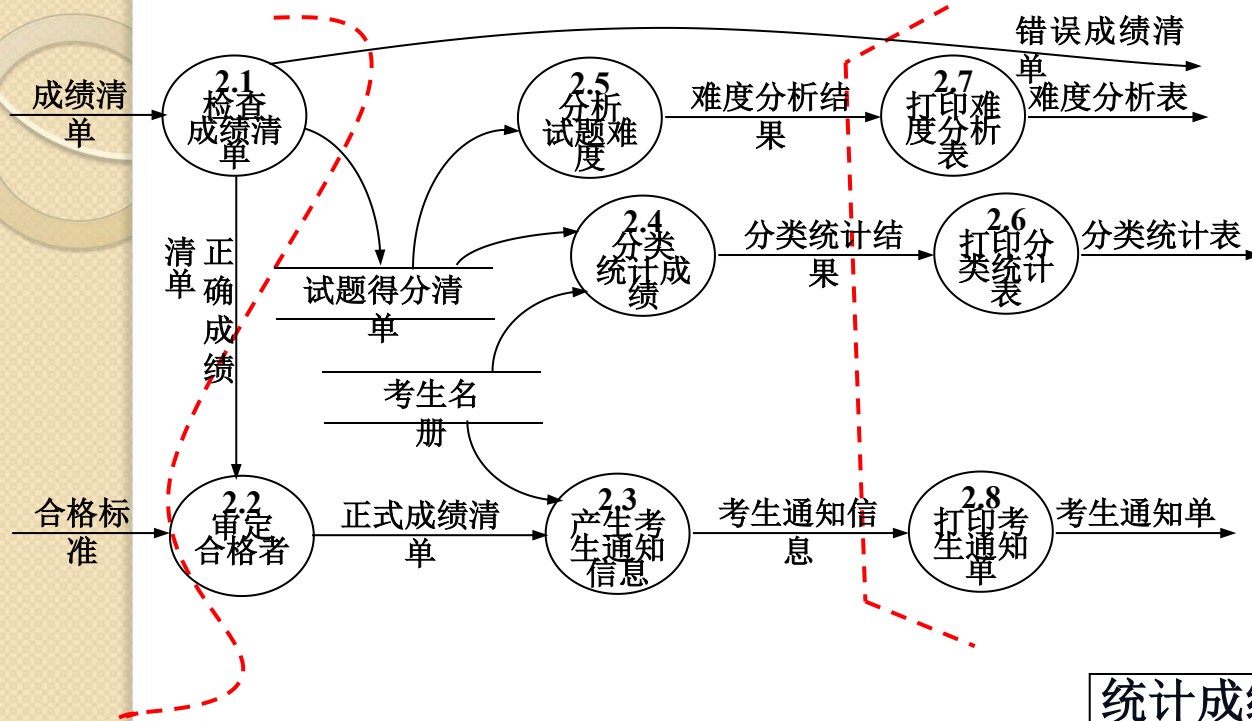
## 4.3 典型的软件架构风格

- (2) 调用/返回风格
- 3种典型的调用/返回风格：
  - 主程序/子程序
  - 面向对象风格
  - 层次结构
- 风格变种
  - B/S结构
  - C/S结构

## 4.3.2 主程序/子程序风格

- 20世纪70年代，结构化程序设计方式出现，主程序/子程序结构(Main Program/Subroutines) 成为结构化设计的一种典型风格。
  - 该架构风格从功能的观点设计系统
  - 通过逐步分解和逐步细化得到系统架构，即将大系统分解为若干模块(模块化)，主程序调用这些模块实现完整的系统功能
  - 主程序的正确性依赖于它所调用的子程序的正确性。

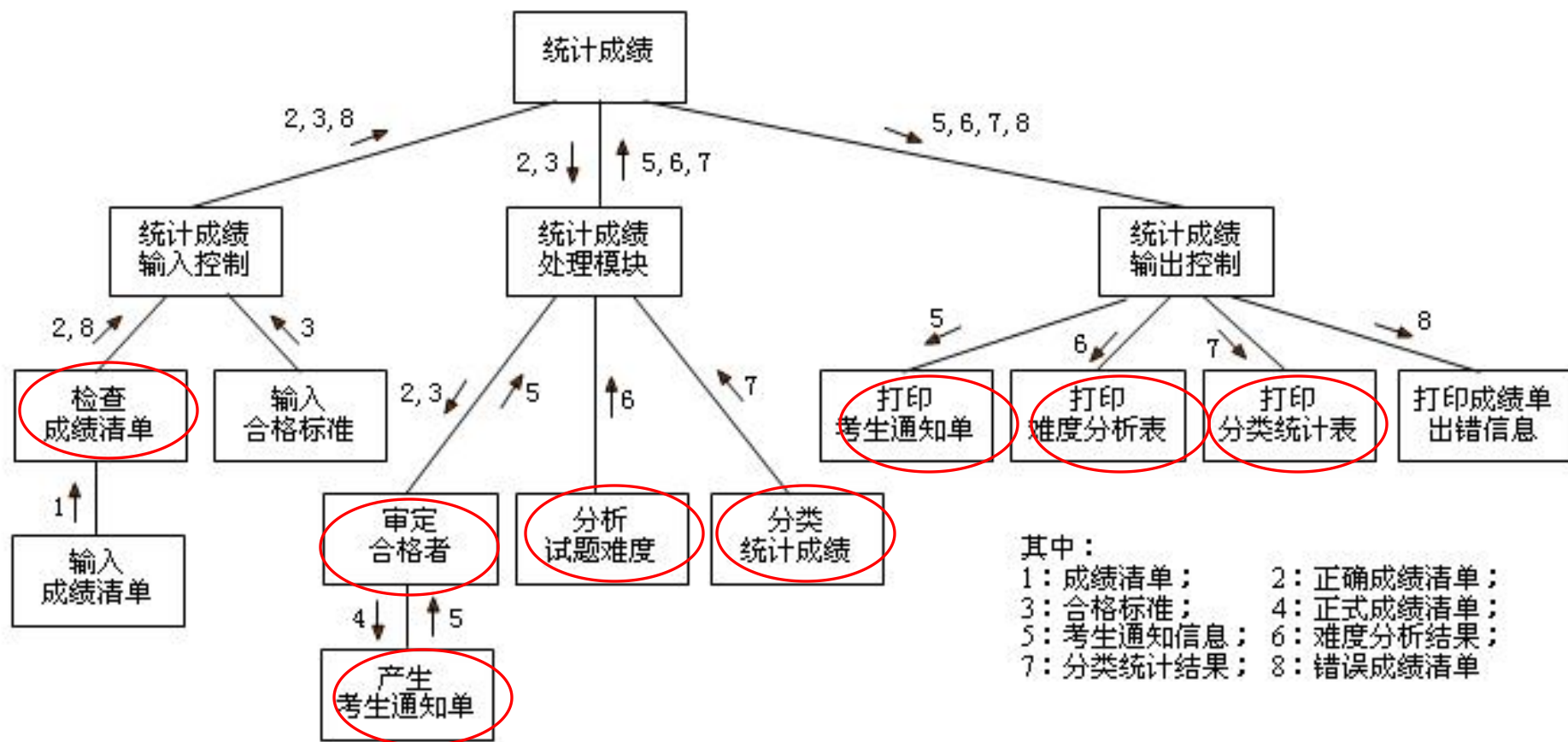
# 示例：统计成绩子图的第一级分解



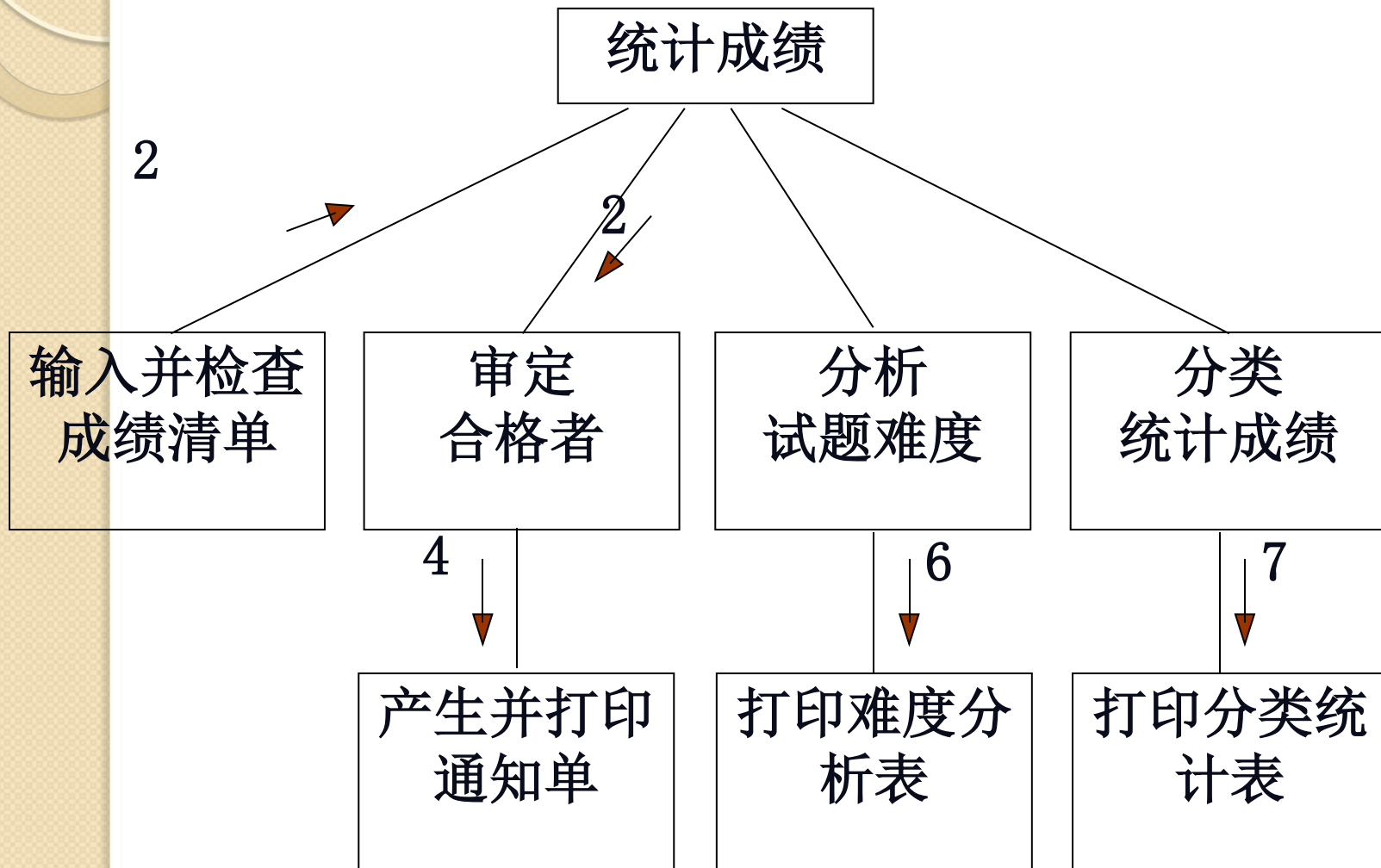
“统计成绩”第一级分解的结构图



# “统计成绩”第二级分解的结构图



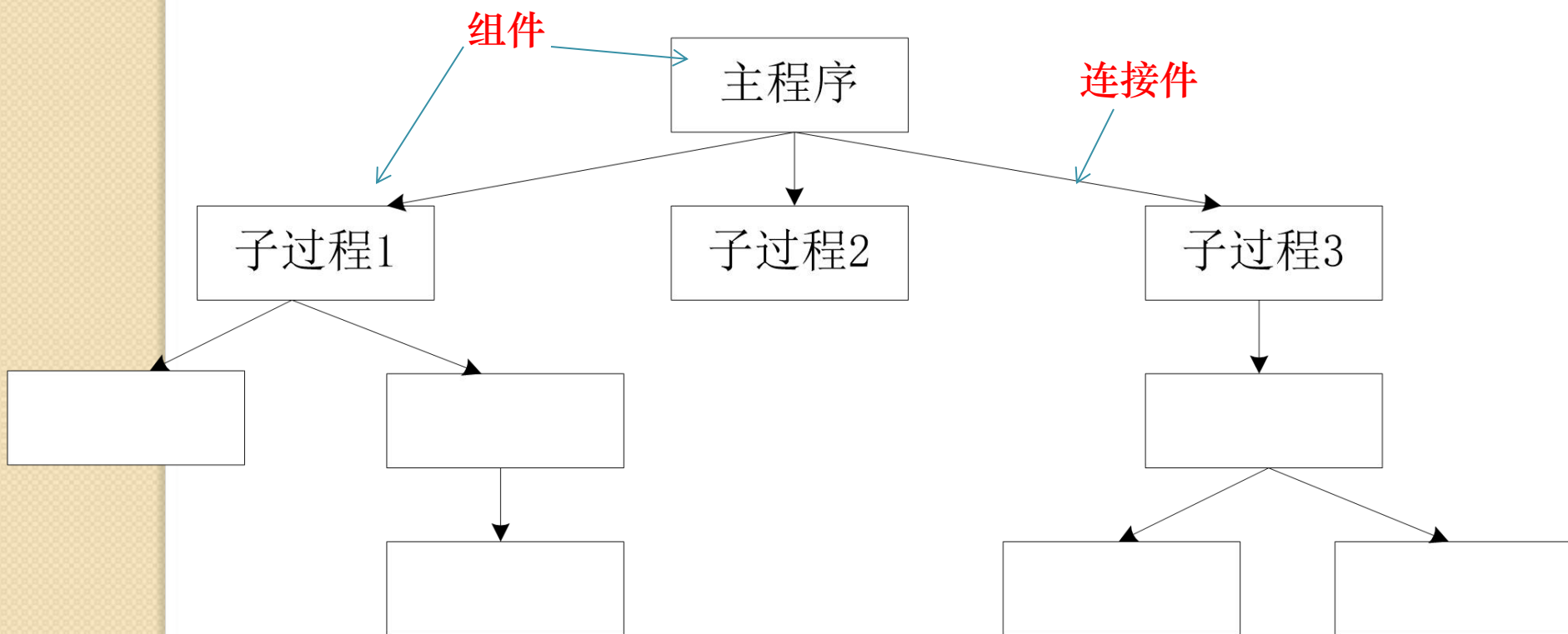
# “统计成绩” 结构图改进





## 4.3.2 主程序/子程序风格

- 其中，组件为**主程序和子程序**，连接件为**调用-返回机制**，拓扑结构为层次化结构。



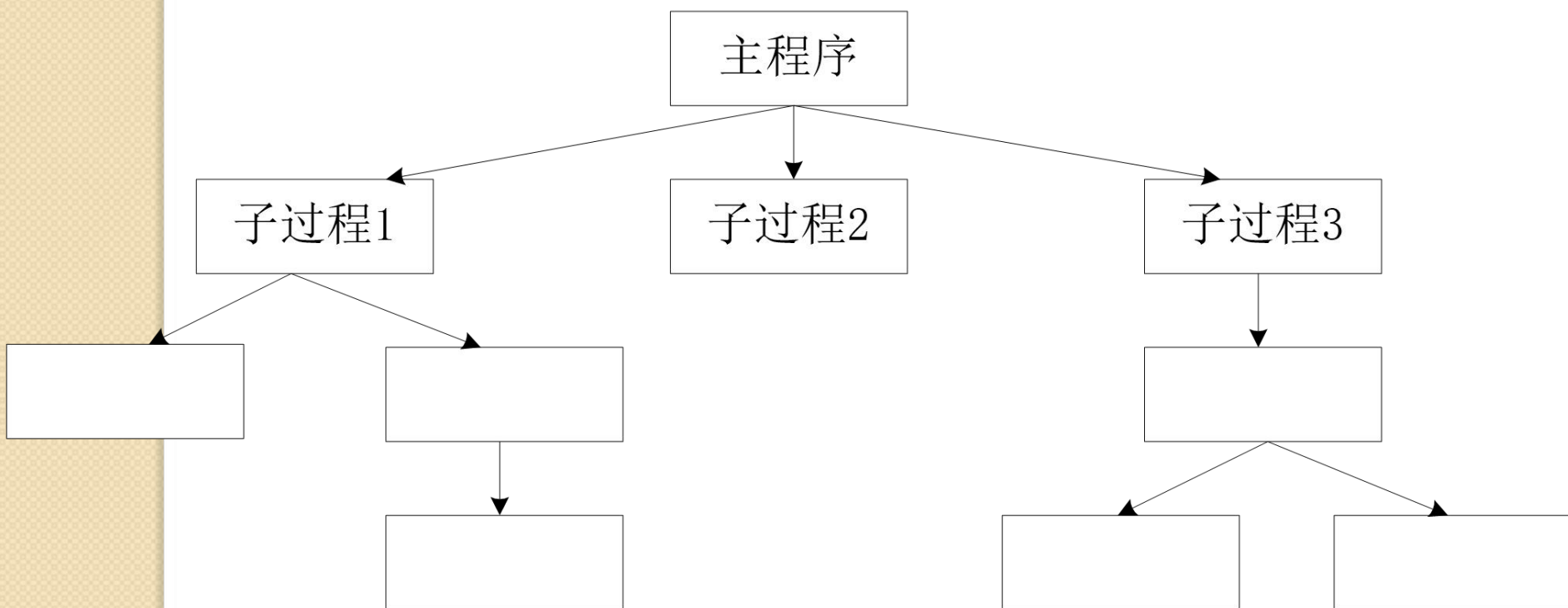
## 4.3.2 主程序/子程序风格

- 应用场景： It is suitable for applications in which the computation can appropriately be defined via a hierarchy of procedure definitions.
  - Many programming languages provide natural support for defining nested collections of procedures and for calling them hierarchically. These languages often allow collections of procedures to be grouped into modules, thereby introducing name-space locality. The execution environment usually provides a single thread of control in a single name space

## 4.3.2 主程序/子程序风格

- 系统模型

- call and definition hierarchy, subsystems often defined via modularity



## 4.3.2 主程序/子程序风格

- 基本组件

- procedures and explicitly visible data
- 程序=数据结构+算法

- 连接件

- procedure calls and explicit data sharing (显式数据共享)

- 约束

- single thread

## 4.3.2 主程序/子程序风格

- 优缺点分析

- 优点：

- (1) 结构化程序设计的典型风格，相对于非结构化设计逻辑清晰，易理解。
    - (2) 开发过程采用逐步细化，将大系统分解为若干模块（模块化）。

## 4.3.2 主程序/子程序风格

- 优缺点分析

- 缺点：

- (1) 对数据存储格式的变化将会影响几乎所有的模块。
    - (2) 结构化程序在规模变大时会难理解、难测试、难维护。
    - (3) 这种分解方案难以支持有效的复用。随着程序规模的增大，大量函数、变量之间的关系错综复杂，要抽取可重用的代码往往变得十分困难。

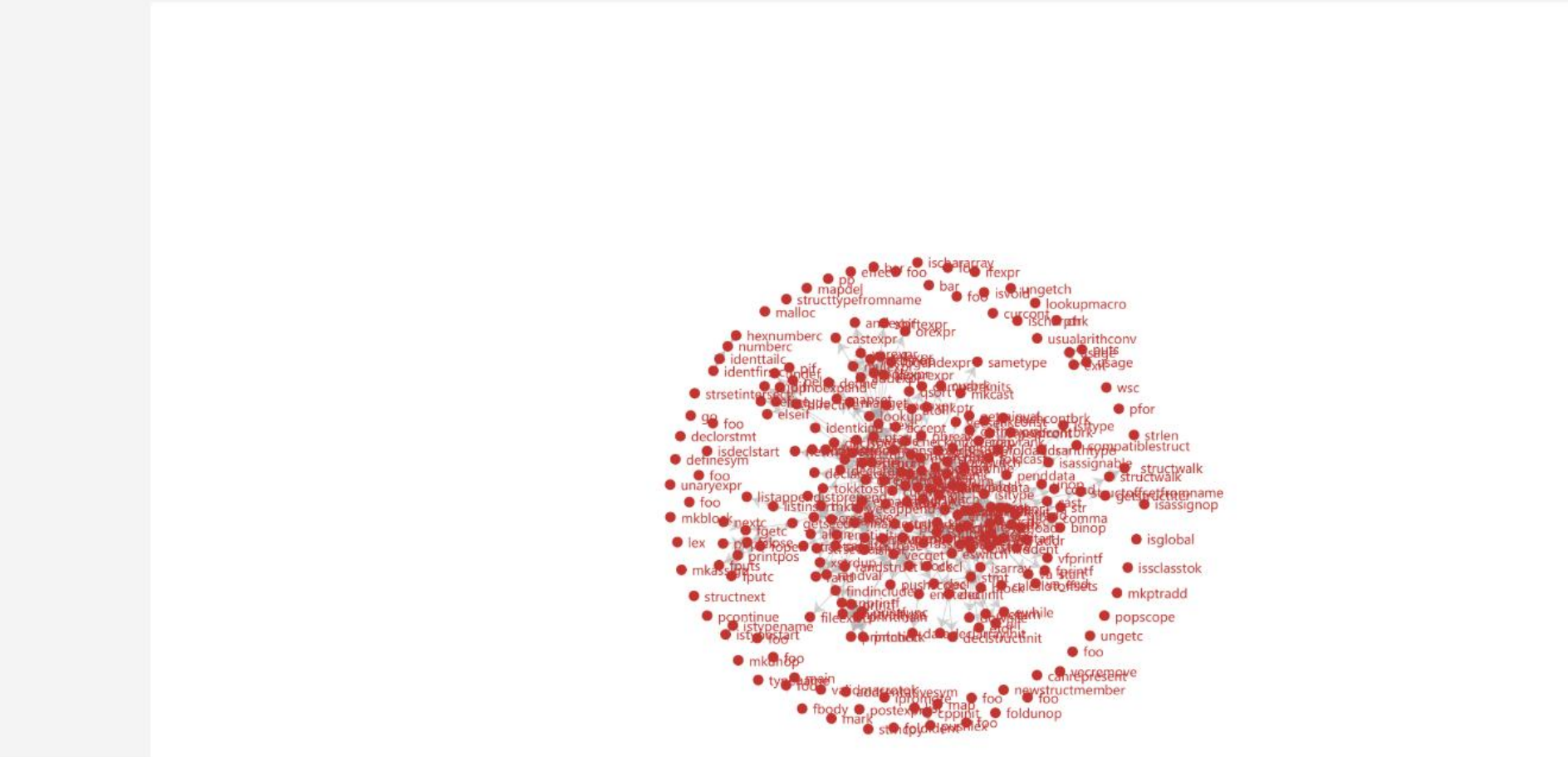


函数调用图 ×

文件调用图

▼

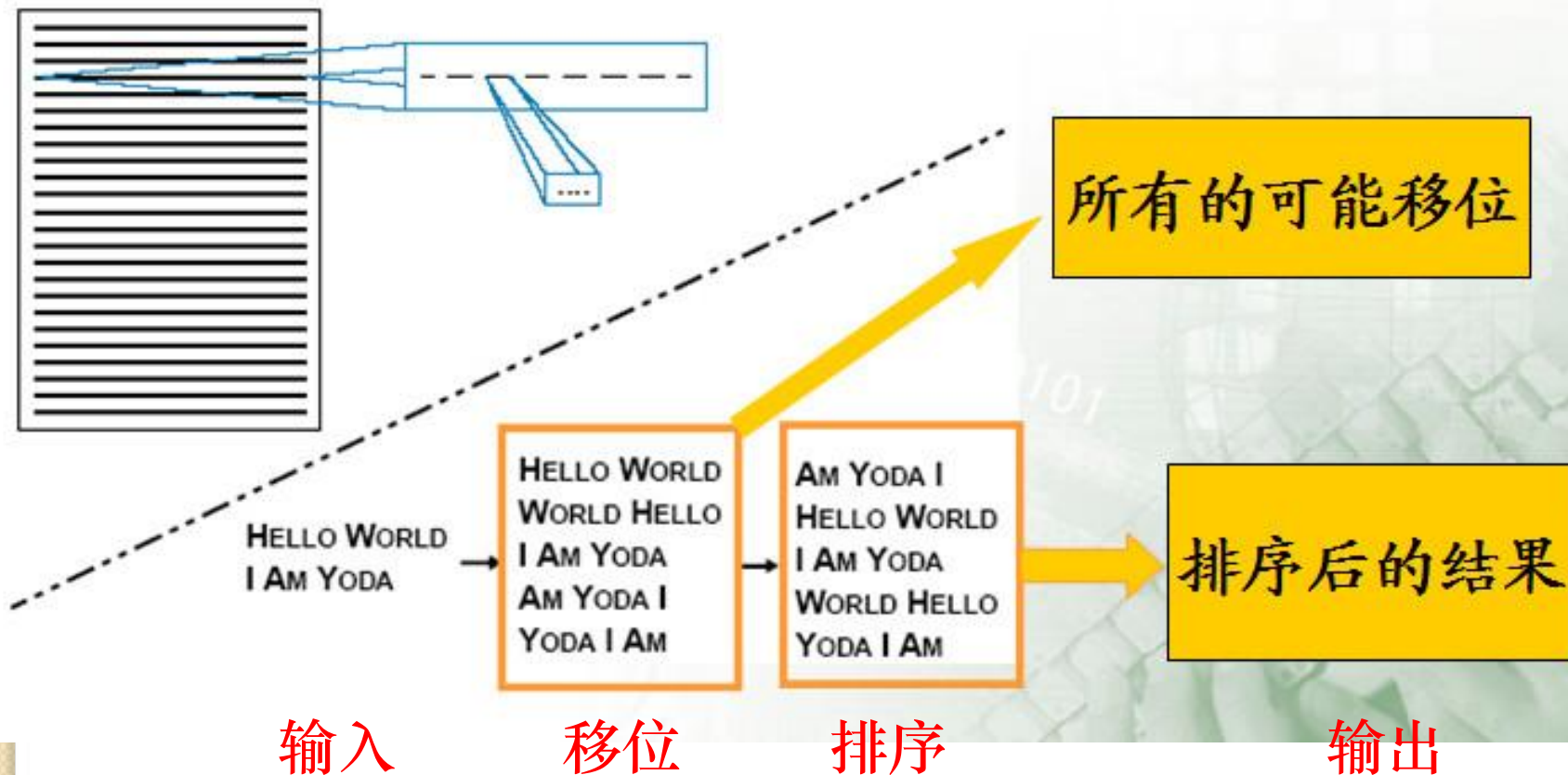
选择



## 4.3.2 主程序/子程序风格

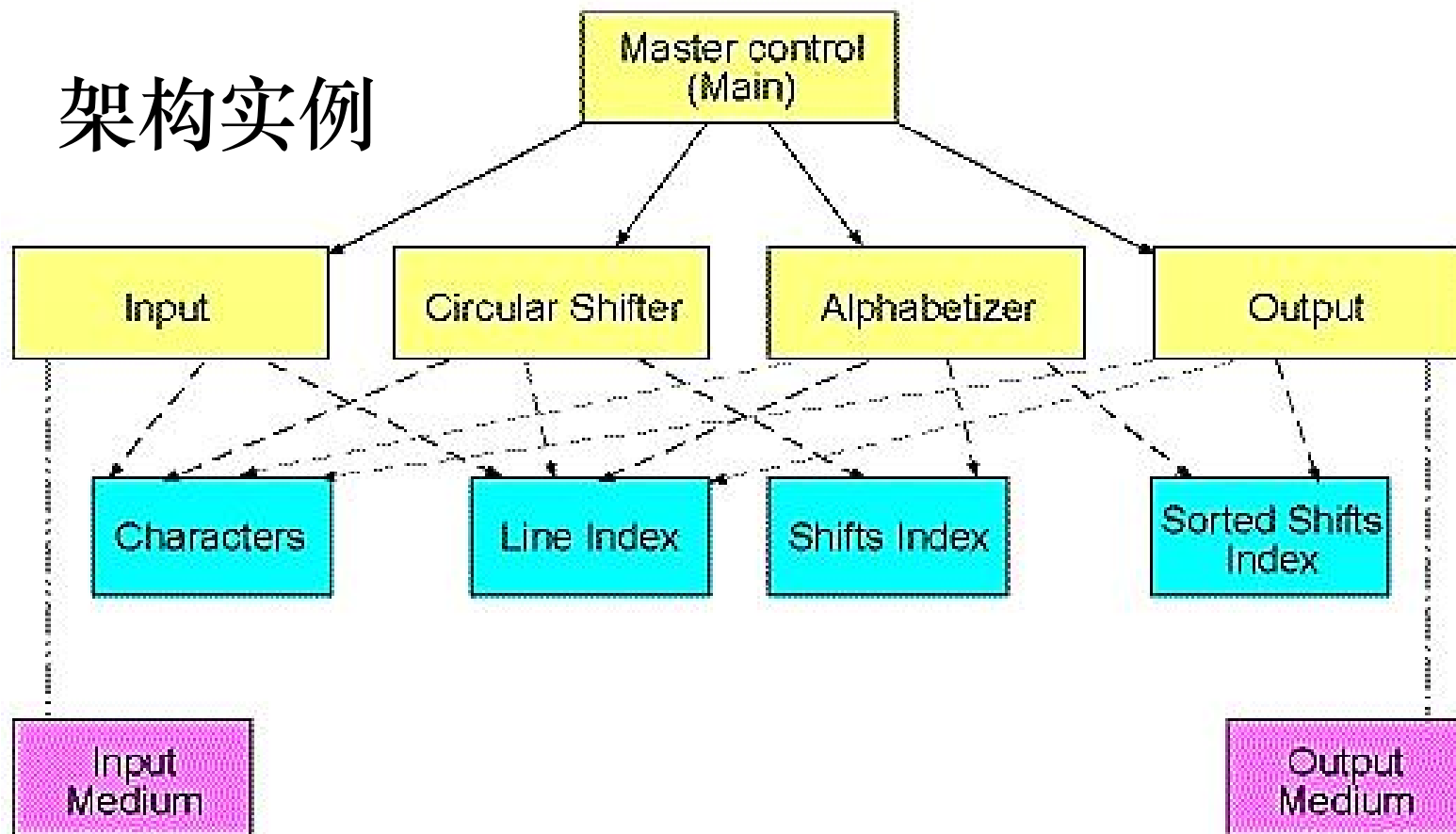
- 应用实例
  - KWIC (Key Word in Context) KWIC作为一个早年间在ACM的Paper提出的一个问题，被全世界各个大学的软件设计课程奉为课堂讲义或者作业的经典。

问题描述：检索系统接受有序的行集合；每一行是单词的有序集合；每一个单词又是字母的有序集合。通过重复地删除行中第一个单词并把它插入到行尾，每一行可以被“循环地移动”。KWIC检索系统以字母表的顺序输出一个所有行循环移动的列表。



## 4.3.2 主程序/子程序风格

### 架构实例



Legend:

- |                                                                                                          |                                                                                                            |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
|  Modules (Functions)  |  Direct Memory Access |
|  Data Repr. in Memory |  Subprogram Call      |
|  I/O Medium           |  System I/O           |

# 作业

- 思考题：
  - KWIC还可以采用哪些风格？分析各自的架构特点和运行特点。

## 4.3 典型的软件架构风格

- (2) 调用/返回风格
- 3种典型的调用/返回风格：
  - 主程序/子程序
  - 面向对象风格
  - 层次结构



## 4.3.3 面向对象风格

- Parnas (1972) :
  - **Hide secrets** (not just representations)
  - Try to localize future change
  - Use functions to allow for change
- Booch (1980+) :
  - Object's behavior is characterized by actions that it suffers and that it requires

## 4.3.3 面向对象风格

- 面向对象（Object-Oriented）方法是80年代初期提出的一种新型的程序设计方法，它彻底改变了过去数据流、事物流分析方式的缺点，采用直接对问题域进行自然抽象的方法，并逐渐发展成包括面向对象分析、设计、编程、测试、维护等一整套内容的完整体系。
- 该架构风格从现实世界中客观存在的事物出发，强调直接以问题域中的事物为中心的来思考问题、认识问题，根据这些事物的本质特征，将其抽象为系统中的对象，并作为系统中的基本构成单位。



# 中国软件大会(CCF ChinaSoft)

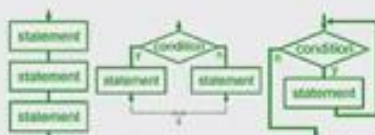
第20届全国软件与应用学术会议  
The 20th National Software and Application Conference  
第6届全国形式化方法与应用会议  
The 6th National Conference on Formal Method and Application

## 程序设计分解粒度的结构化演进

### 指令 (Instruction)

(运算、循环、条件分支、跳转语句)

程序 = 指令序列的控制与组合



### 过程 (Procedure)

(过程、例程、子程序、函数等)

程序 = 过程的调用与组合



### 对象 (Object)

(可接收、处理并传递数据的独立实体)

程序 = 对象的组合与数据交互



低

抽象层次

高

第20届全国软件与应用学术会议

The 20th National Software and Application Conference

第6届全国形式化方法与应用会议

The 6th National Conference on Formal Method and Application

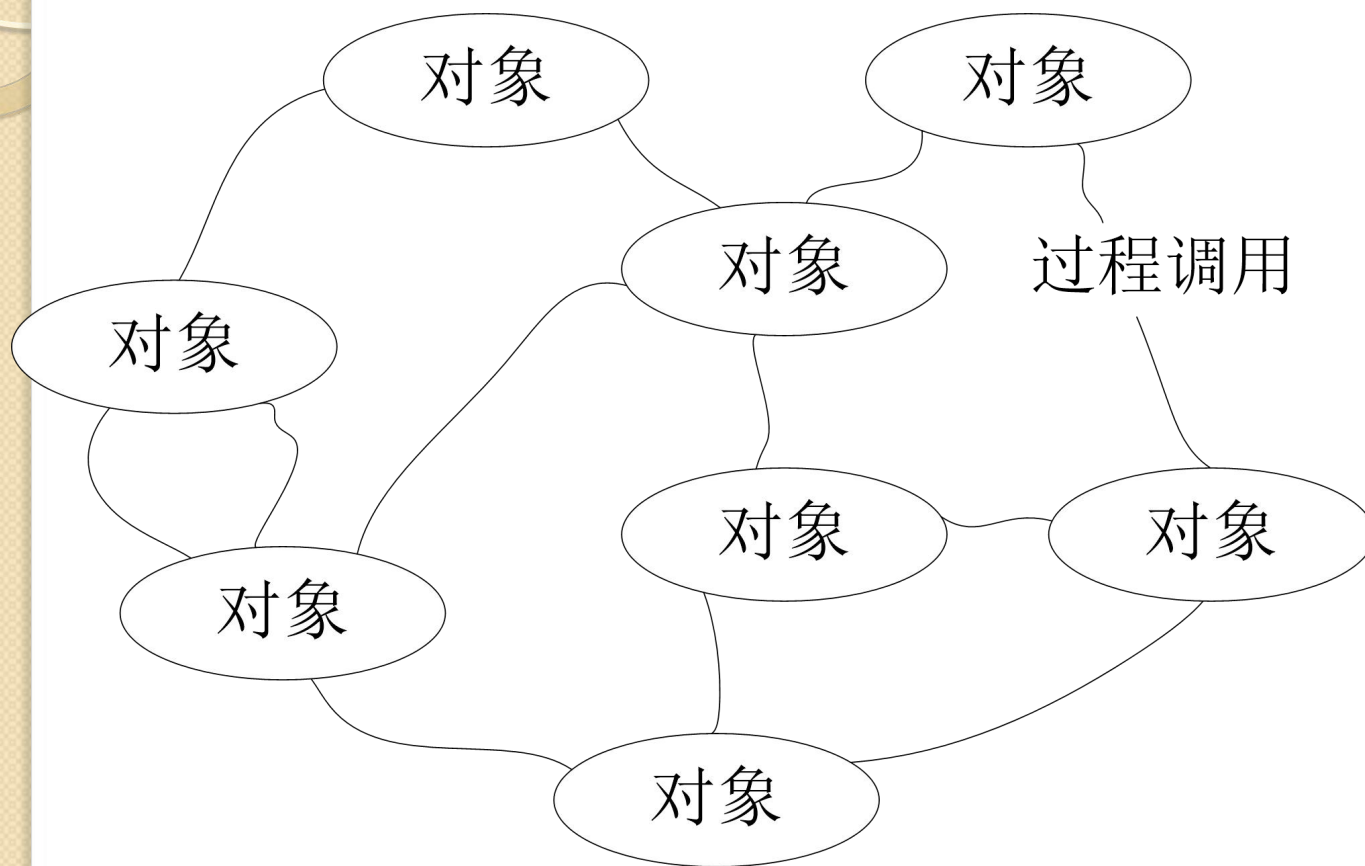
2021年12月5日



孙凝晖

中国工程院院士

### 4.3.3 面向对象风格



### 图4-7面向对象风格



## 4.3.3 面向对象风格

- 面向对象组织结构有两个重要特点：
  - (1)对象负责维护其表示的完整性（通常是通过保持其表示上的一些不变式来实现的）；
  - (2)对象的表示对其他对象而言是隐蔽的。抽象数据类型的使用，以及面向对象系统的使用已经非常普遍。

### 4.3.3 面向对象风格

- 应用场景： It is suitable for applications in which a central issue is **identifying and protecting** related bodies of information, especially representation information.
- 系统模型： localized state maintenance
- 组件： managers (e.g., servers, objects, abstract data types)
- 连接件： procedure call
- 约束： decentralized, usually single thread



## 4.3.3 面向对象风格

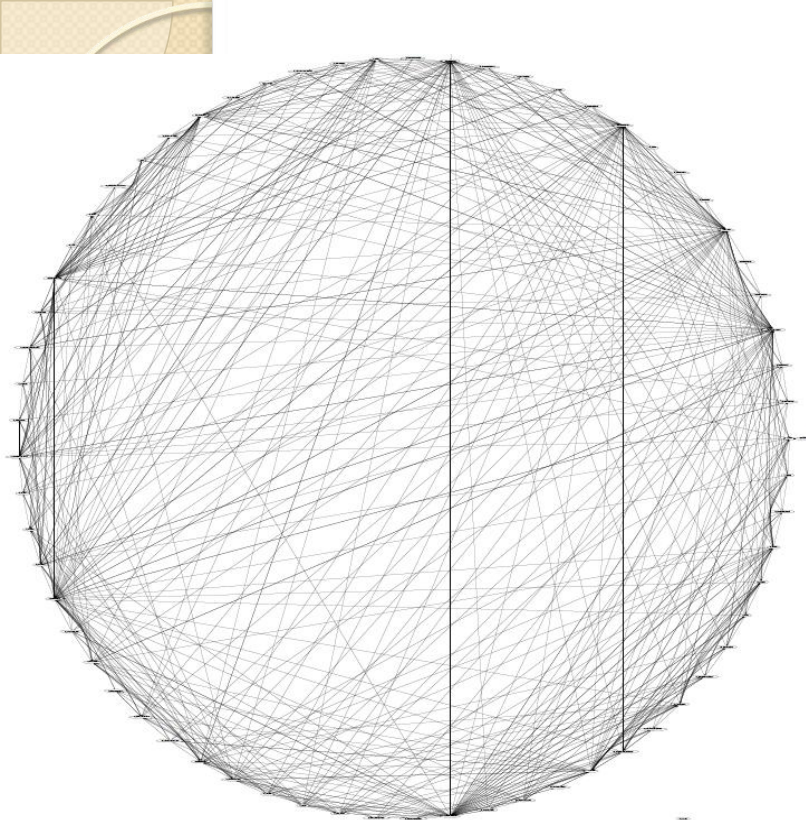
- 优点：

- (1) 对象隐藏了其实现细节，所以可以在不影响其它对象的情况下改变对象的实现，不仅使得对象的使用变得简单、方便，而且具有很高的安全性和可靠性。
- (2) 设计者可将一些数据存取操作的问题分解成一些交互的代理程序的集合。

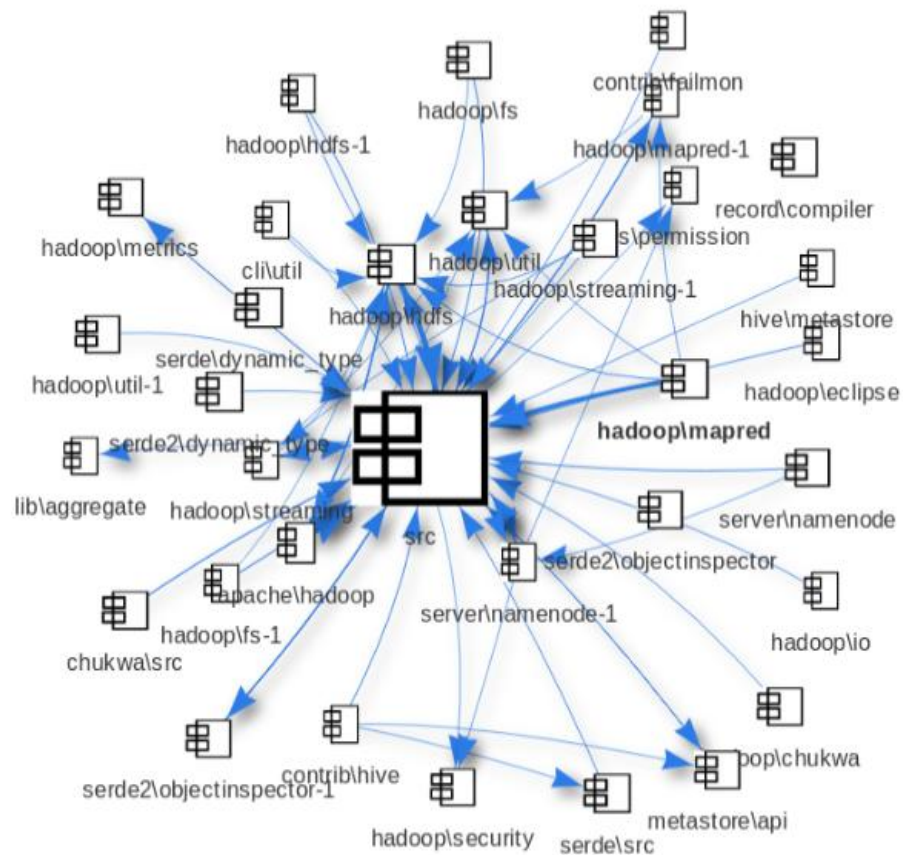
## 4.3.3 面向对象风格

- 缺点

- Managing many objects
  - Vast sea of objects requires additional structuring
- Managing many interactions
  - Single interface can be limiting & unwieldy (hence, “friends”)
  - Some languages/systems permit multiple interfaces (inner class, interface, multiple inheritance)
- Distributed responsibility for behavior
  - Makes system hard to understand
  - Interaction diagrams now used in design
- Capturing families of related designs
  - Types/classes are often not enough
  - Design patterns as an emerging off-shoot



## Hadoop-0.19的真实架构



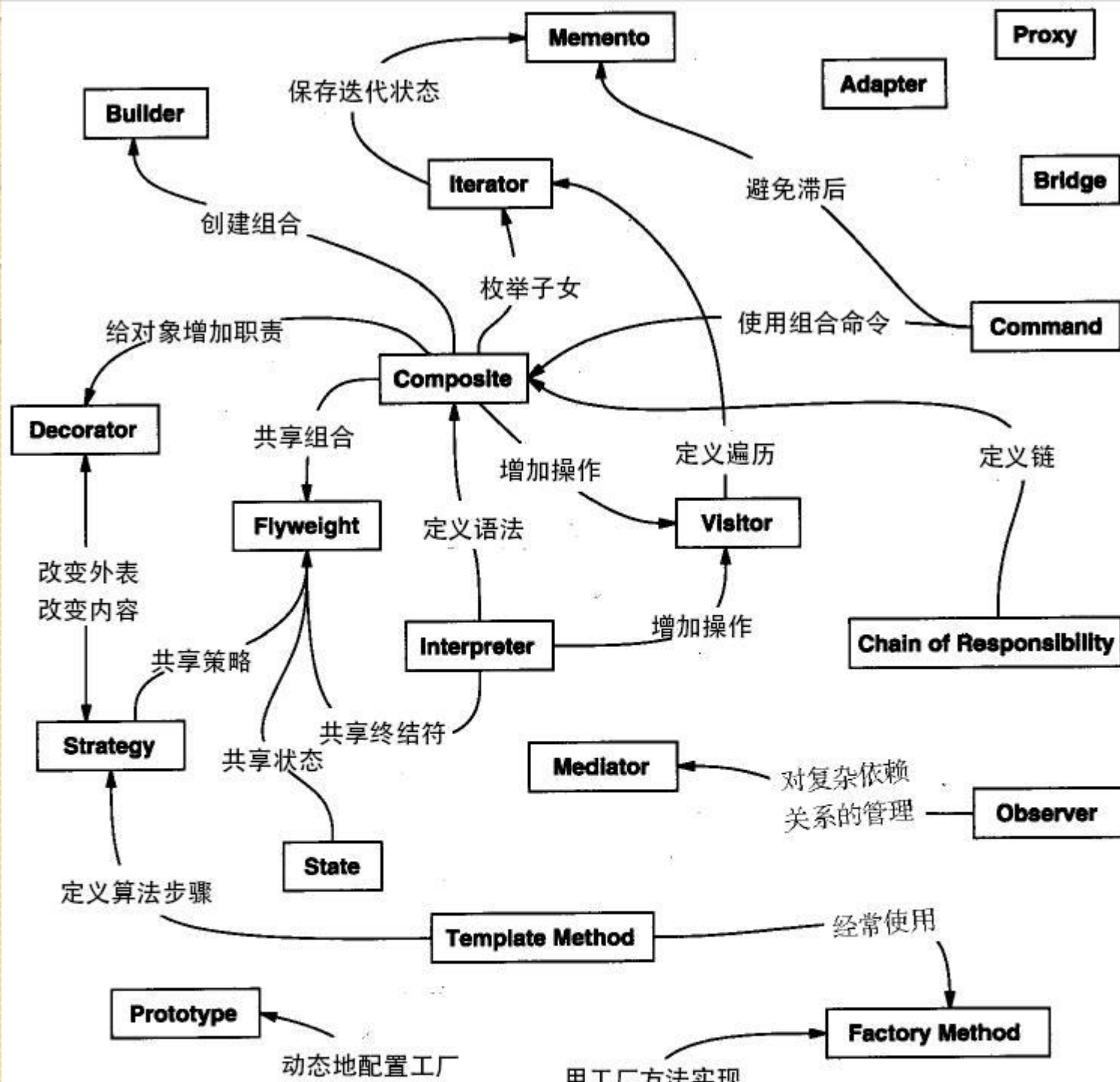
## Hadoop-0.19的恢复架构

## 4.3.3 面向对象风格

- 设计模式：
  - 参考书 Design Patterns - Elements of Reusable Object-Oriented Software（中文译名：设计模式 - 可复用的面向对象软件元素）
  - 23 种设计模式，这些模式可以分为三大类：创建型模式（Creational Patterns）、结构型模式（Structural Patterns）、行为型模式（Behavioral Patterns）



| 序号 | 模式 & 描述                                                                                          | 包括                                                                                                                                                                                                                                                                                                                           |
|----|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <b>创建型模式</b><br>这些设计模式提供了一种在创建对象的同时隐藏创建逻辑的方式，而不是使用 new 运算符直接实例化对象。这使得程序在判断针对某个给定实例需要创建哪些对象时更加灵活。 | <ul style="list-style-type: none"><li>● 工厂模式 (Factory Pattern)</li><li>● 抽象工厂模式 (Abstract Factory Pattern)</li><li>● 单例模式 (Singleton Pattern)</li><li>● 建造者模式 (Builder Pattern)</li><li>● 原型模式 (Prototype Pattern)</li></ul>                                                                                                 |
| 2  | <b>结构型模式</b><br>这些设计模式关注类和对象的组合。继承的概念被用来组合接口和定义组合对象获得新功能的方式。                                     | <ul style="list-style-type: none"><li>● 适配器模式 (Adapter Pattern)</li><li>● 桥接模式 (Bridge Pattern)</li><li>● 过滤器模式 (Filter、Criteria Pattern)</li><li>● 组合模式 (Composite Pattern)</li><li>● 装饰器模式 (Decorator Pattern)</li><li>● 外观模式 (Facade Pattern)</li><li>● 享元模式 (Flyweight Pattern)</li><li>● 代理模式 (Proxy Pattern)</li></ul> |
| 3  | <b>行为型模式</b><br>这些设计模式特别关注对象之间的通信。                                                               | <ul style="list-style-type: none"><li>● 责任链模式 (Chain of Responsibility Pattern)</li><li>● 命令模式 (Command Pattern)</li><li>● 解释器模式 (Interpreter Pattern)</li><li>● 迭代器模式 (Iterator Pattern)</li><li>● 中介者模式 (Mediator Pattern)</li><li>● 备忘录模式 (Memento Pattern)</li><li>● 观察者模式 (Observer Pattern)</li></ul>                  |





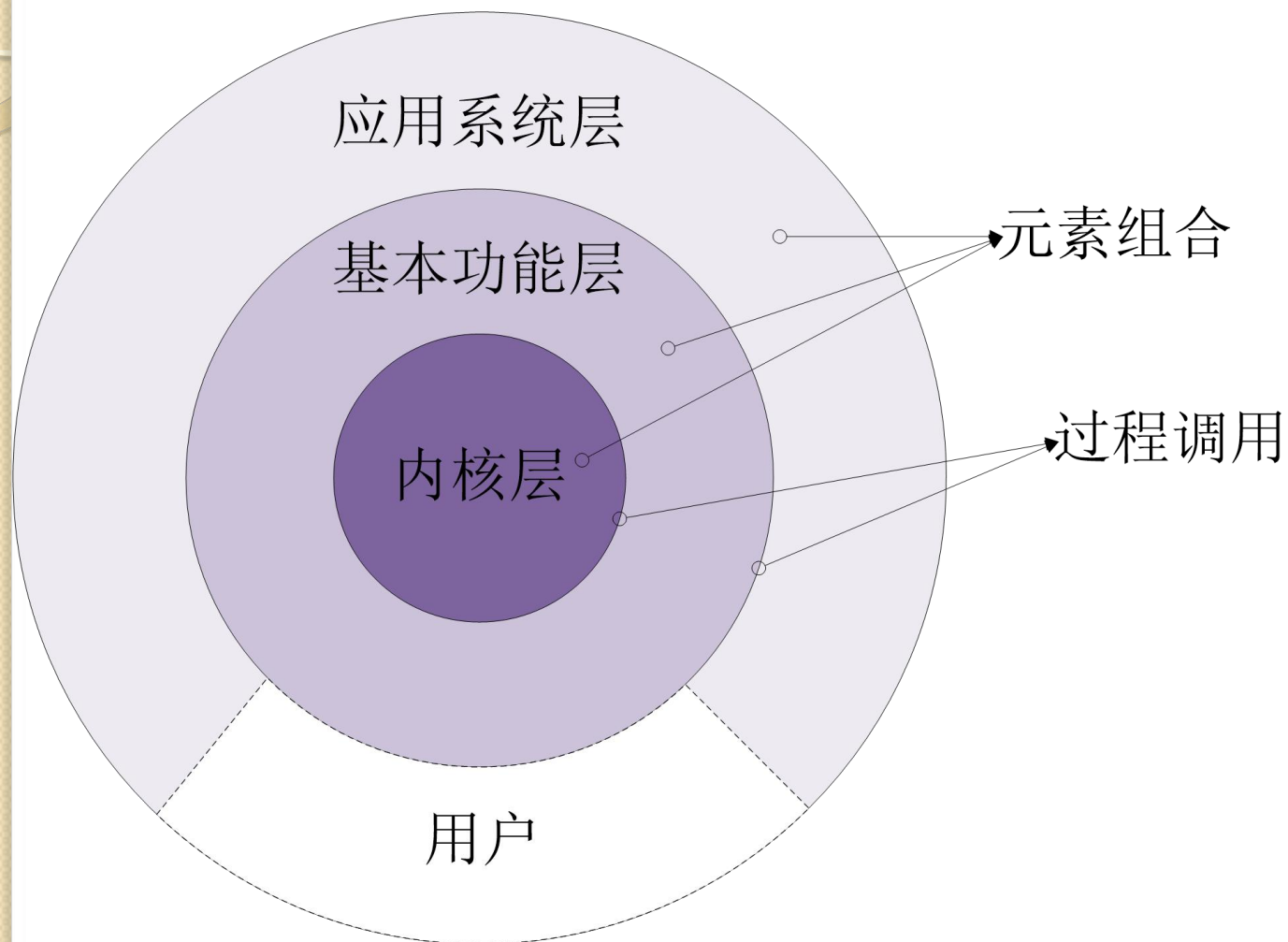
## 4.3 典型的软件架构风格

- (2) 调用/返回风格
- 3种典型的调用/返回风格：
  - 主程序/子程序
  - 面向对象风格
  - 层次结构

## 4.3.4 层次化风格

- 基本思想
  - 在分层系统（Layered System）中，系统被组织成若干个层次，每个层次由一系列组件组成；层次之间存在接口，通过接口形成call/return的关系——下层组件向上层组件提供服务，上层组件被看作是下层组件的客户端。
- 层次化早已经成为一种复杂系统设计的普遍性原则。

## 4.3.4 层次化风格



## 4.3.4 层次化风格

- 在分层架构中，组件被划分成几个水平层，每个层在应用中执行特定角色。
- 大多数分层架构由四个标准层组成：
  - 表现层 (presentation)
  - 业务层 (business)
  - 持久层 (persistence)
  - 数据库层 (database)
  - (有些情况下将业务层和持久层结合在一起成为一个单独的业务层)

### 4.3.3 层次化风格

- 应用场景： It is suitable for applications that involve distinct classes of services that can be arranged hierarchically.
- 系统模型： hierarchy of opaque（不透明的）layers
- 组件： usually composites; composites are most often collections of procedures
- 连接件： depends on structure of components; often procedure calls under restricted visibility（在受限条件下可见）
- 约束： single thread



## 4.3.3 层次化风格

- 分层架构特点

- 分层架构中的每层为上一层提供服务，使用下一层的服务，只能见到与自己邻接的层。
- 即：当请求从一个层移动到另一个层时，它必须经过它正下方的层，以到达该层下面的下一层。
- 例如，源自表现层的请求必须首先经过业务层，然后在最终到达数据库层之前到达持久层。

## 4.3.3 层次化风格

- 分层架构特点
  - 大的问题分解为若干个渐进的小问题，逐步解决，隐藏了很多复杂度
  - 修改一层，最多影响两层，而通常只能影响上层。接口稳固，则谁都不影响
  - 上层必须知道下层的身份，不能调整层次之间的顺序

## 4.3.4 层次化风格

- 优点:
  - (1) 支持基于可增加抽象层的设计，允许将一个复杂问题分解成一个增量步骤序列的实现。
  - (2) 支持扩展。每一层的改变最多只影响相邻层。
  - (3) 支持重用。只要给相邻层提供相同的接口，它允许系统中同一层的不同实现相互交换使用。

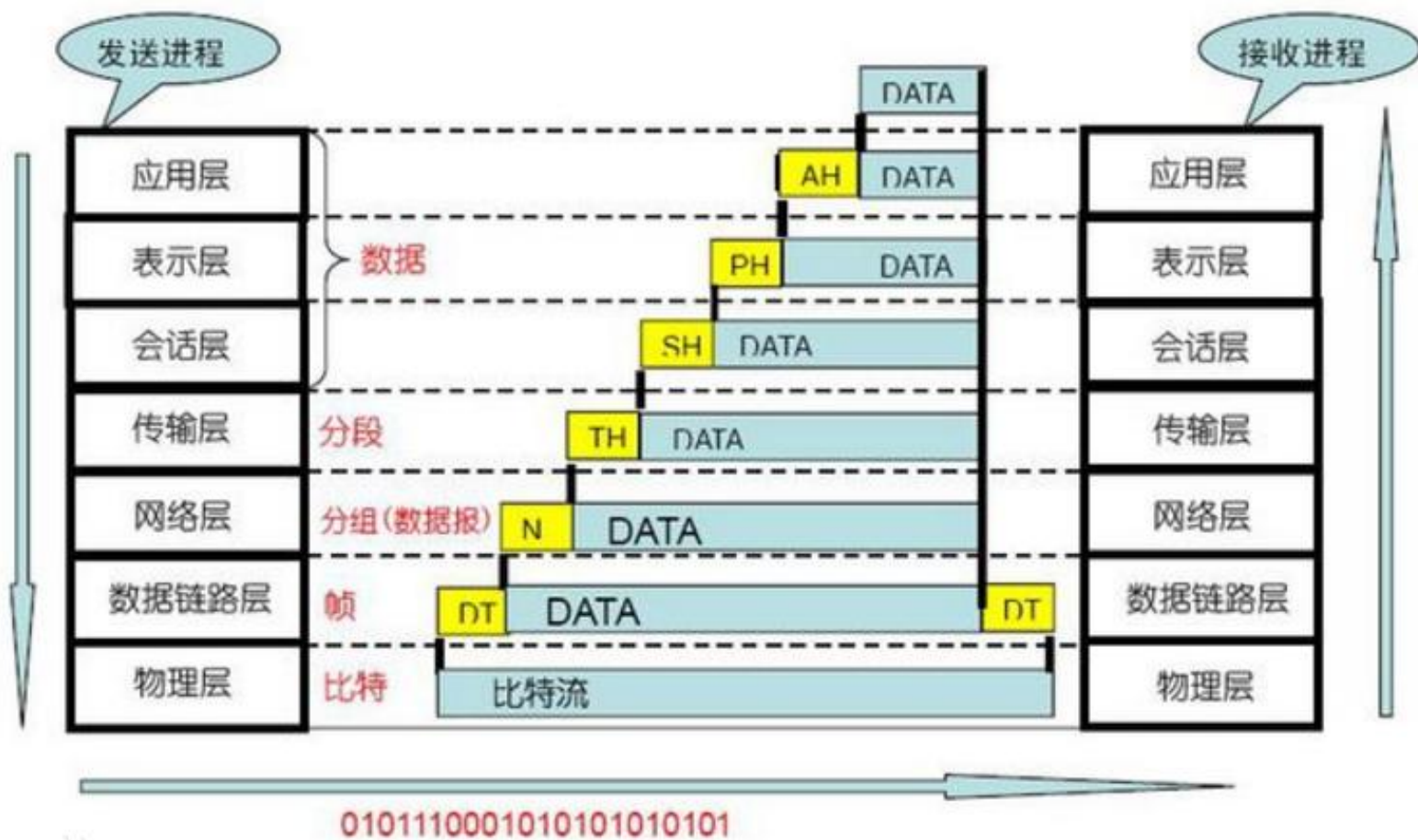
## 4.3.4 层次化风格

- 缺点

- (1) 不是所有系统都容易用这种模式来构建；
- (2) 定义一个合适的抽象层次可能会非常困难，特别是对于标准化的层次模型。
  - 例如，实际的通信协议体就很难映射到ISO框架中，因为其中许多协议跨多个层。
- (3) 层层相调，影响性能

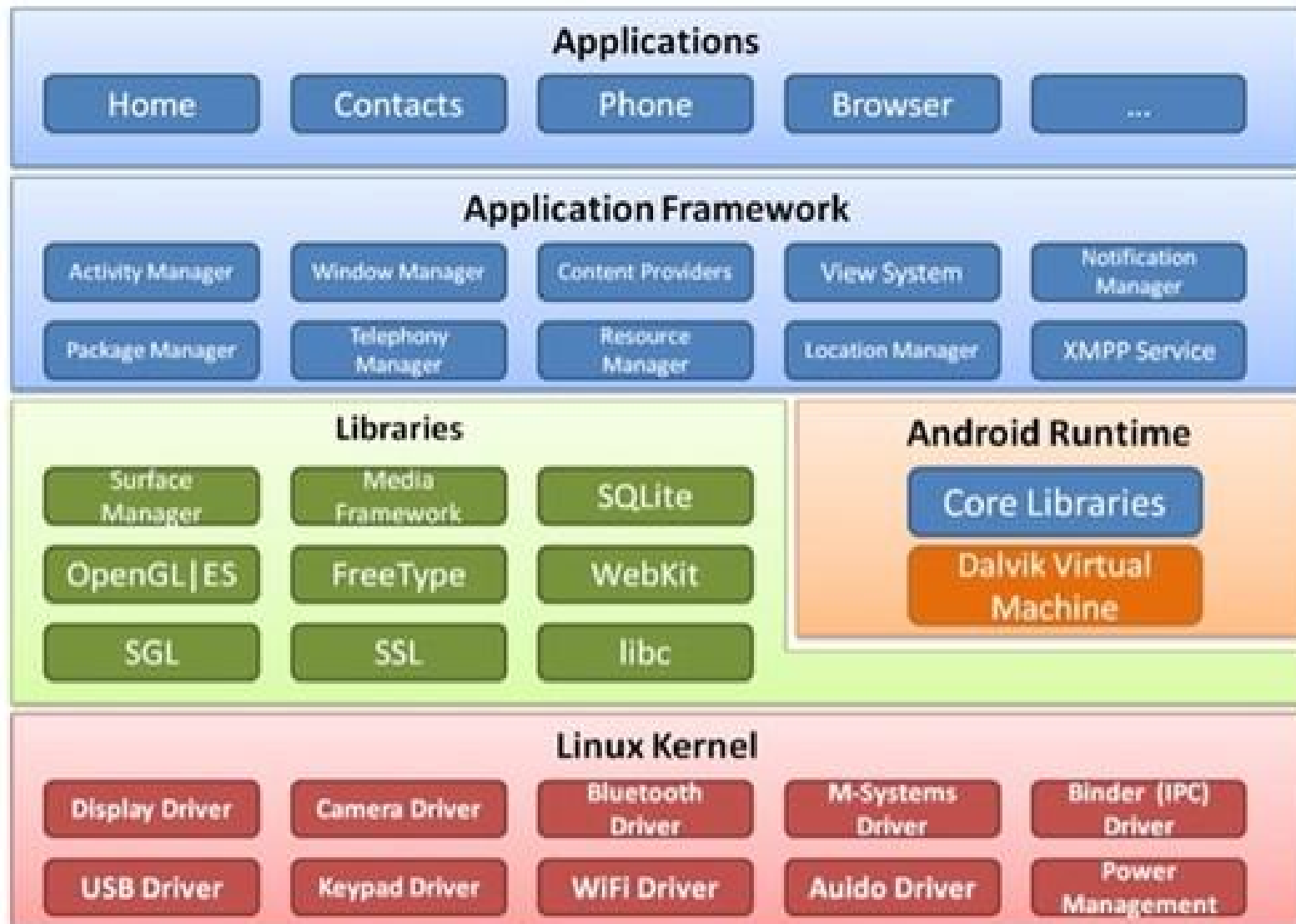
## 4.3.4 层次化风格

- 应用实例：http分层通信协议





# 安卓系统的层次结构



## 4.3 典型的软件架构风格

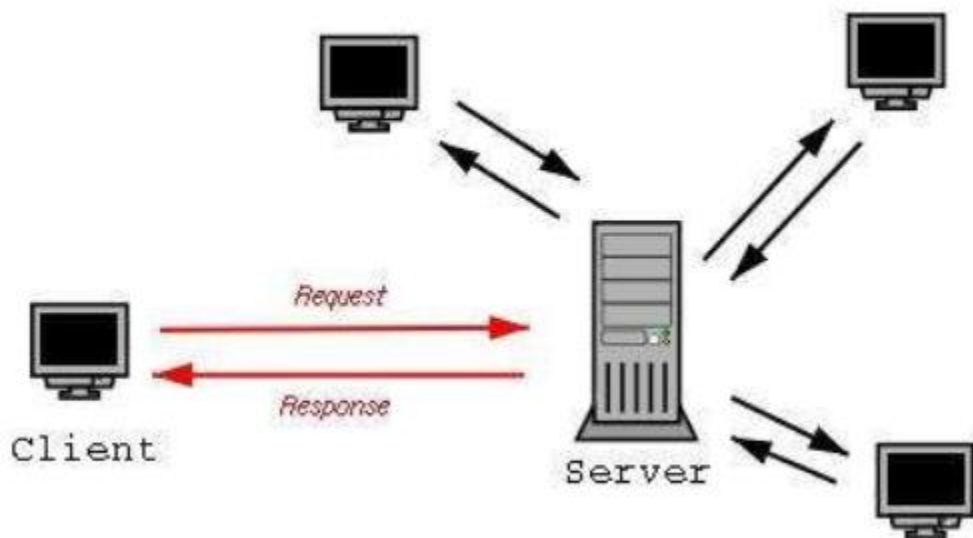
- (2) 调用/返回风格
- 3种典型的调用/返回风格：
  - 主程序/子程序
  - 面向对象风格
  - 层次结构
- 风格变种
  - 客户机/服务器风格
  - 浏览器/服务器风格

## 4.3.11 客户机/服务器风格

- 客户机/服务器 (Client/Server)是20世纪90年代开始成熟的一项技术，主要针对资源不对等问题而提出的一种共享策略。
- 客户机和服务器是两个相互独立的逻辑系统，为了完成特定的任务而形成一种协作关系。
  - 客户机(前端，front-end)：业务逻辑、与服务器通讯的接口；
  - 服务器(后端：back-end)：与客户机通讯的接口、业务逻辑、数据管理。

## 4.3.11 客户机/服务器风格

- 一般的，客户机为完成特定的工作向服务器发出请求；服务器处理客户机的请求并返回结果。
- 客户机程序和服务器程序配置在同一台计算机上时：采用消息、共享存储区和信号量等方法实现通信连接。
- 客户机程序和服务器程序配置在分布式环境中时：通过远程调用（Remote Produce Call, RPC）协议来进行通信。



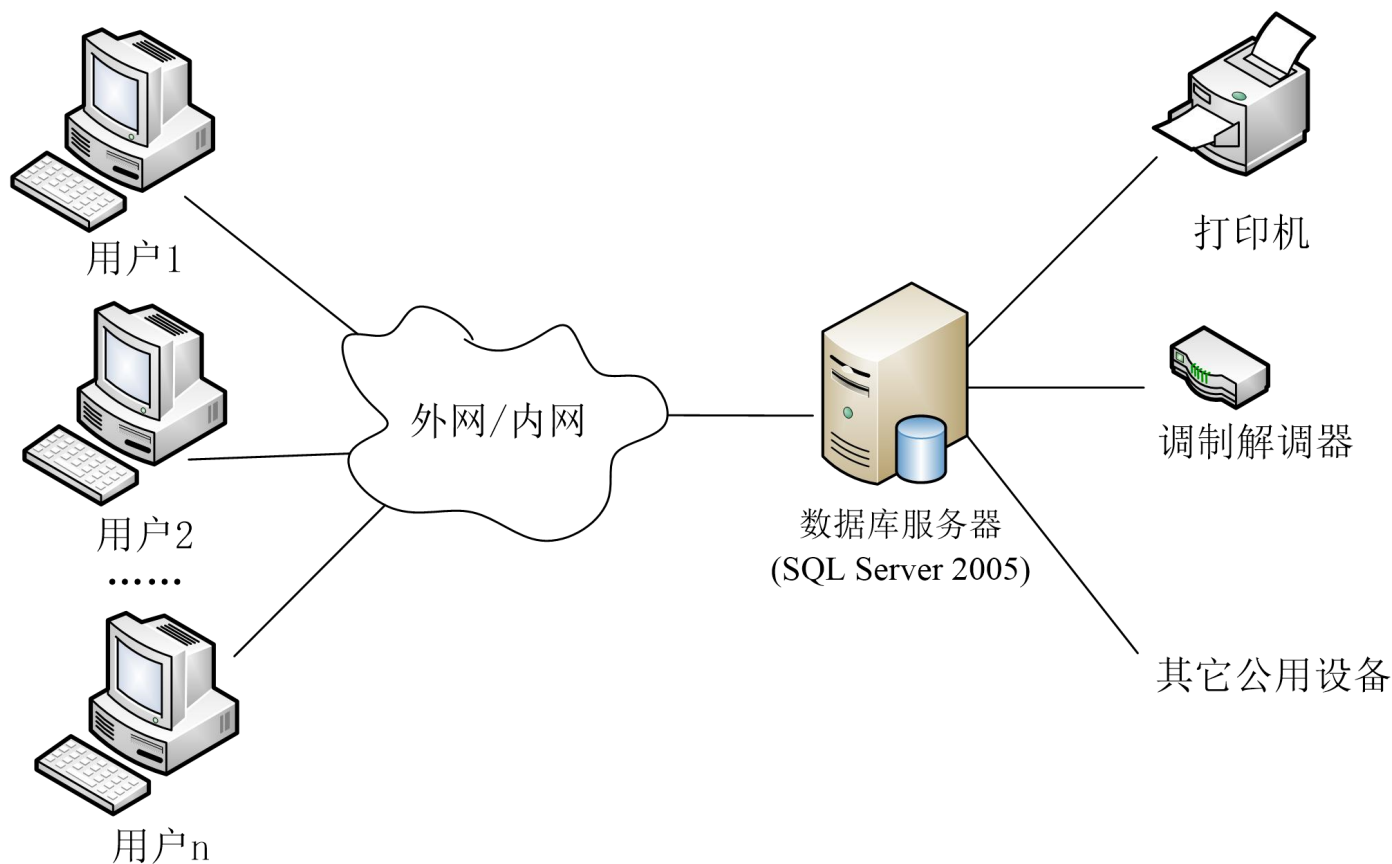
## 4.3.11 客户机/服务器风格

- 两层C/S结构
- 三层C/S结构
- B/S结构（浏览器/服务器风格）



## 4.3.11 客户机/服务器风格

- 两层C/S架构



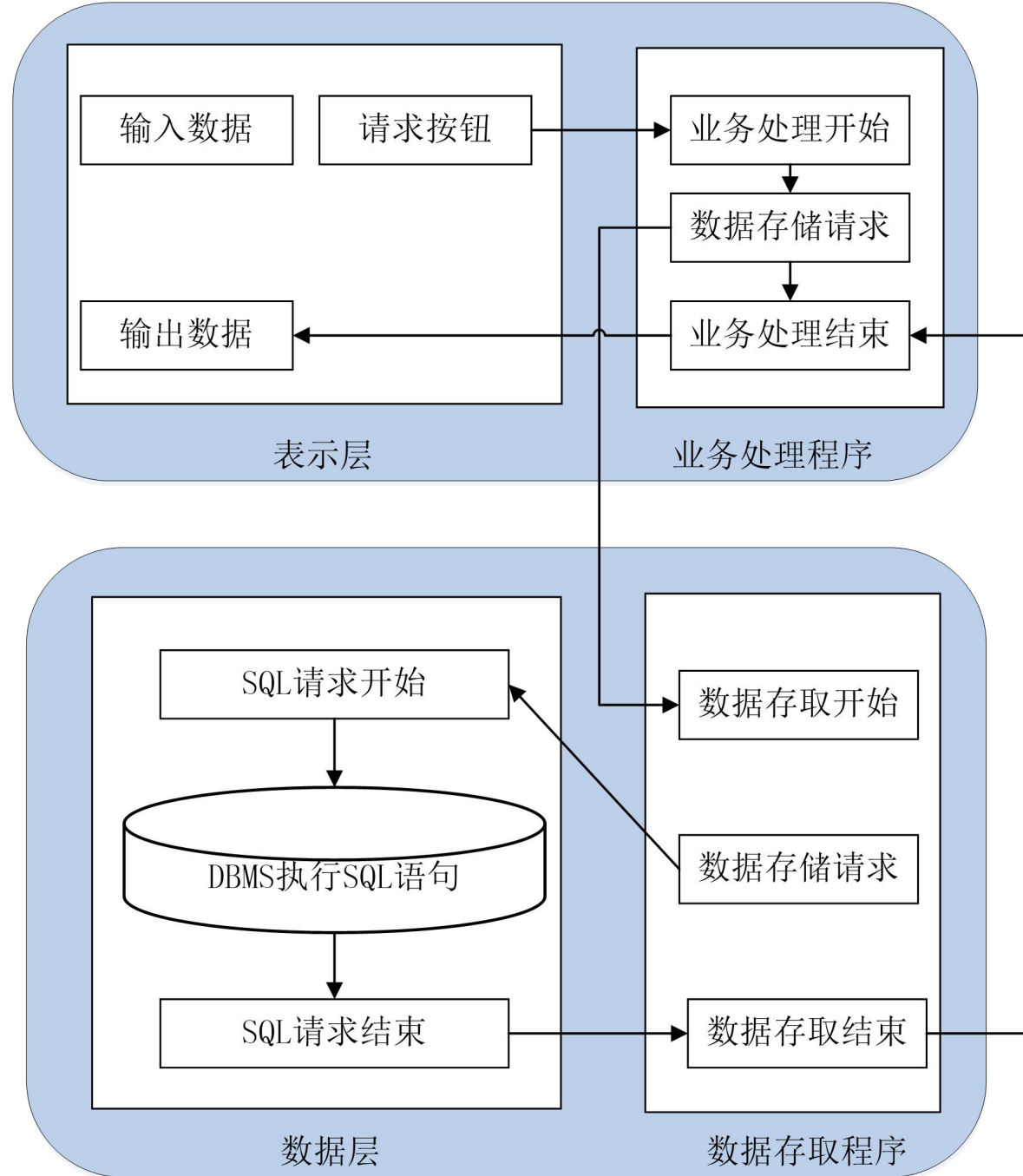


图4-30两层C/S架构的处理流程

## 4.3.11 客户机/服务器风格

- 两层C/S架构优点：

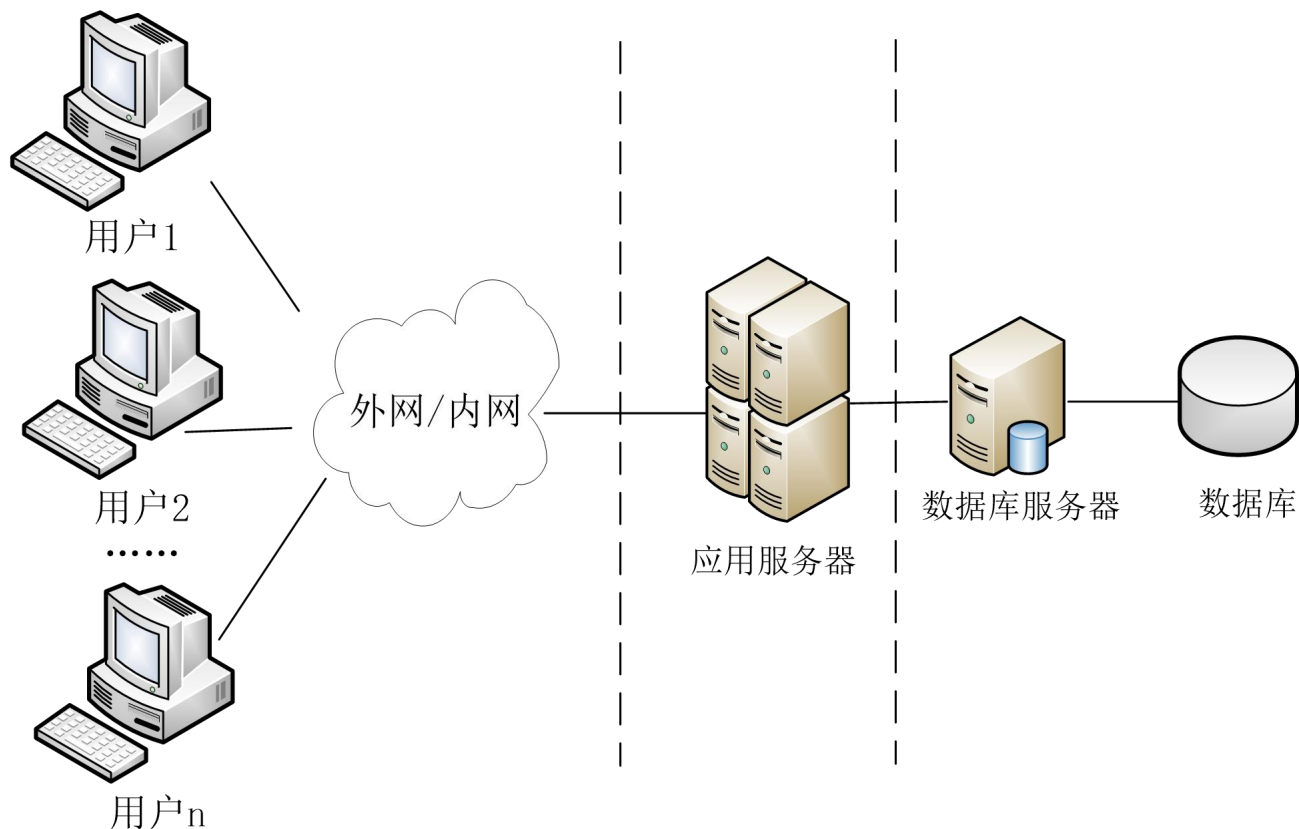
- (1) 客户机组件和服务机组件分别运行在不同的计算机上，有利于分布式数据的组织和处理。
- (2) 组件之间的位置是相互透明的
- (3) 客户机程序和服务程序可运行在不同的操作系统上，便于实现异构环境和多种不同开发技术的融合。
- (4) 软件环境和硬件环境的配置具有极大的灵活性，易于系统功能的扩展。
- (5) 将大规模的业务逻辑分布到多个通过网络连接的低成本的计算机上，降低了系统的整体开销。

## 4.3.11 客户机/服务器风格

- 两层C/S架构缺点：
  - (1) 开发成本较高（客户机的软硬件要求高）。
  - (2) 客户机程序的设计复杂度大，客户机负荷重。
  - (3) 信息内容和形式单一。
  - (4) C/S架构升级需要开发人员到现场更新客户机程序，对运行环境进行重新配置，增加了维护费用。
  - (5) 两层C/S结构采用了单一的服务器，同时以局域网为中心，难以扩展到Internet。
  - (6) 数据安全性不高，客户端程序可以直接访问数据库服务器。

## 4.3.11 客户机/服务器风格

- 三层C/S架构





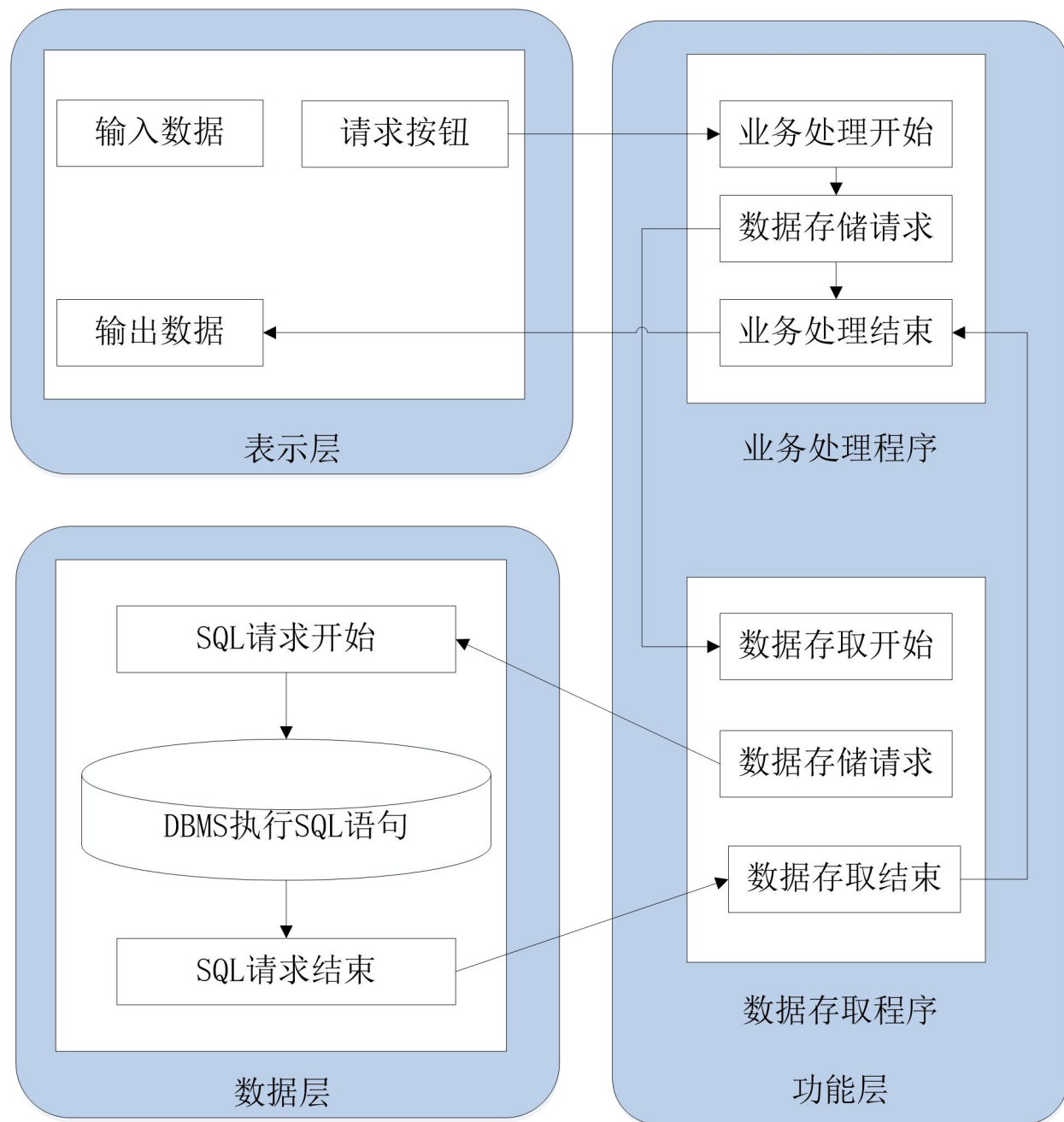


图4-32三层C/S架构的处理流程

## 4.3.11 客户机/服务器风格

- 三层C/S架构相比于两层C/S架构的优点：
  - (1) 合理地划分三层结构的功能，可以使系统的逻辑结构更加清晰，提高软件的可维护性和可扩充性。
  - (2) 在实现三层C/S架构时，可以更有效地选择运行平台和硬件环境，从而使每一层都具有清晰的逻辑结构、良好的负荷处理能力和较好的开放性。
  - (3) 在C/S架构中，可以分别选择合适的编程语言并行开发。
  - (4) 系统具有较高的安全性。

## 4.3.11 客户机/服务器风格

- 在使用三层C/S架构时需注意以下两个问题：
  - (1) 如果各层之间的通信效率不高，即使每一层的硬件配置都很高，系统的整体性能也不会太高。
  - (2) 必须慎重考虑三层之间的通信方法、通信频率和传输数据量，这和提高各层的独立性一样也是实现三层C/S架构的关键性问题。

## 4.3.12 浏览器/服务器风格

- 浏览器/服务器风格是三层C/S风格的一种实现方式。
- 主要包括浏览器，Web服务器和数据库服务器。

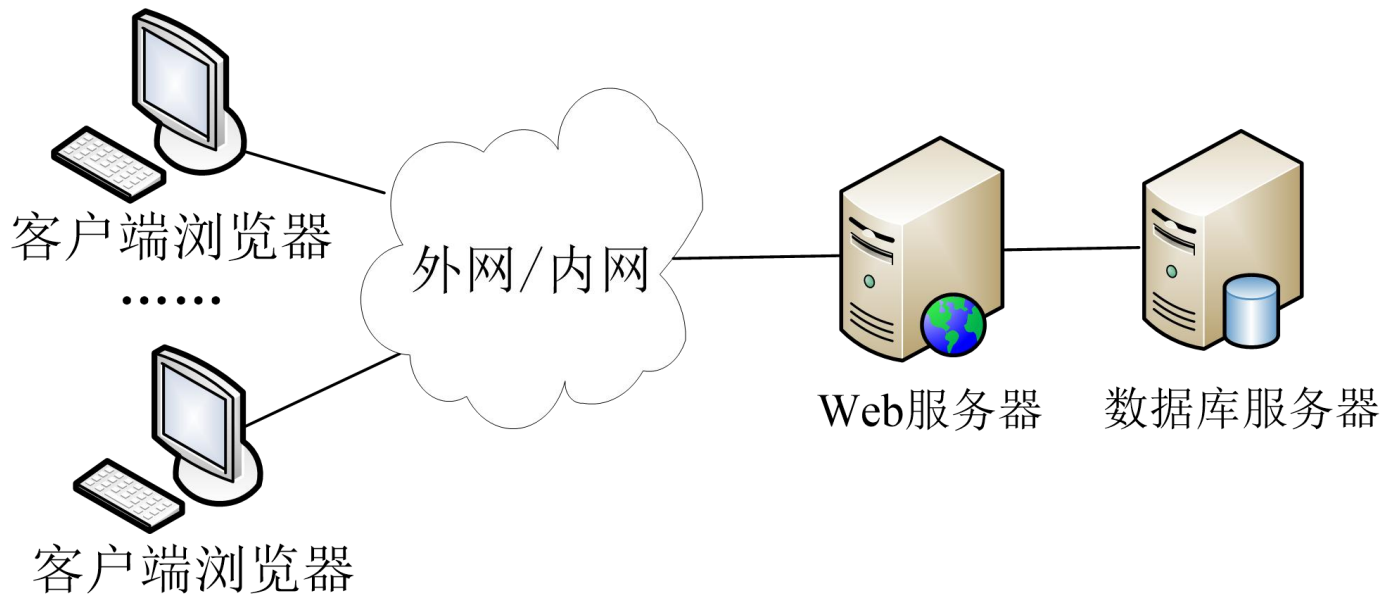


图4.34 B/S架构

## 4.3.12 浏览器/服务器风格

- 与三层C/S结构的解决方案相比，B/S架构在客户机上采用了WWW浏览器，将Web服务器作为应用服务器。
- B/S架构核心是Web服务器，数据请求、网页生成、数据库访问和应用程序执行全部由Web服务器来完成。
- 在B/S架构中，系统安装、修改和维护全在服务器端解决，客户端无任何业务逻辑。



## 4.3.12 浏览器/服务器风格

- 优点

- (1) 客户端只需要安装浏览器，操作简单，能够发布动态信息和静态信息。
- (2) 运用HTTP标准协议和统一客户端软件，能够实现跨平台通信。
- (3) 开发成本比较低，只需要维护Web服务器程序和中心数据库。客户端升级可以通过升级浏览器来实现。

## 4.3.12 浏览器/服务器风格

- 缺点

- (1) 个性化程度比较低，所有客户端程序的功能都是一样的。
- (2) 客户端数据处理能力比较差，加重了Web服务器的工作负担，影响系统的整体性能。
- (3) 在B/S架构中，数据提交一般以页面为单位，动态交互性不强，不利于在线事物处理(Online Transaction Processing, OLTP)。
- (4) B/S架构的可扩展性比较差，系统安全性难以保障。
- (5) B/S架构的应用系统查询中心数据库，其速度要远低于C/S架构。

## 4.3.12 浏览器/服务器风格

- 虽然B/S结构有许多优越性，但是C/S结构起步较早，技术很成熟，网络负载也非常小，因而未来一段时间内将会出现B/S结构和C/S结构共存的现象。
- 然而，计算模式的未来发展趋势将向B/S结构转变。



# COMBAT SUIT

套装-忍者信条

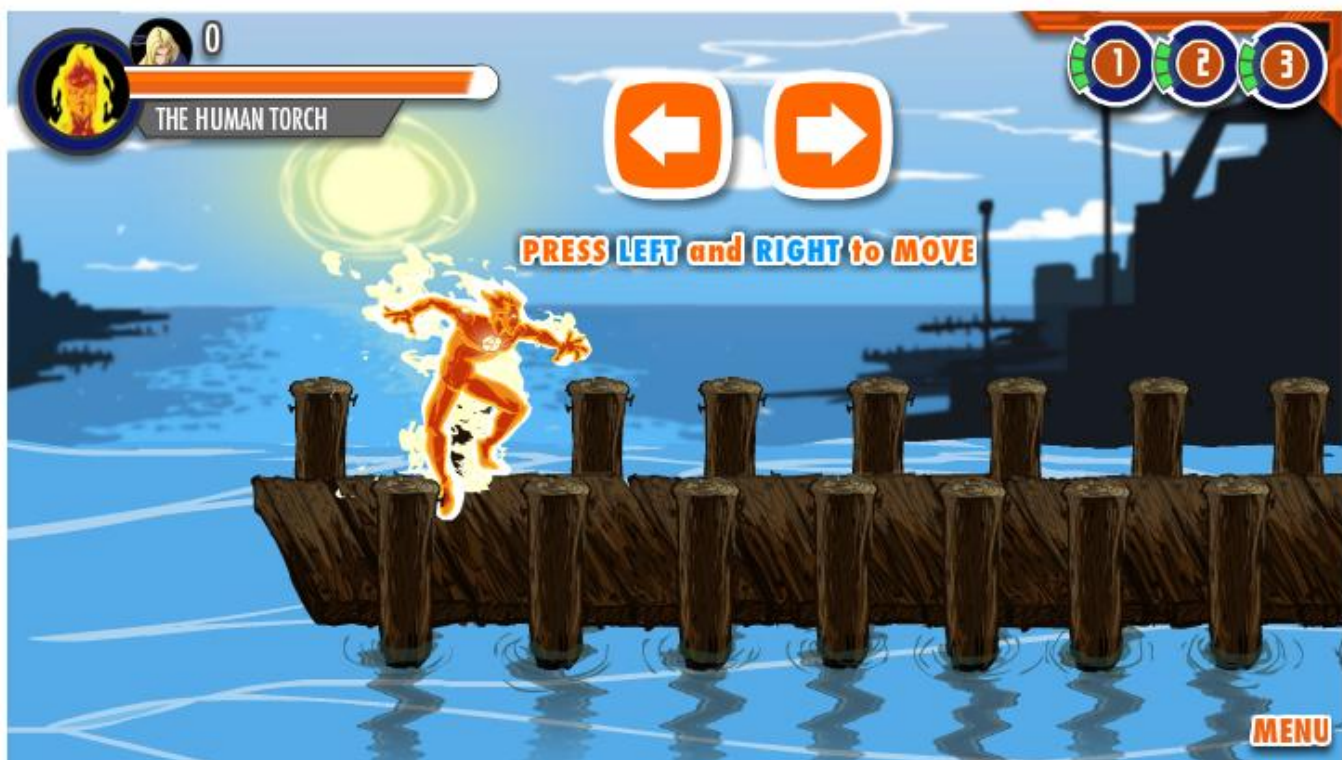
套装-幻影公爵

套装-幻影战神

GAME FOR PEACE  
**和平精英**  
GP.00.COM

神奇四侠冒险之旅无敌版

📱 下载4399手机游戏盒, **十万手游任你玩!**



🔄  
重玩

🖥️  
全屏

📖  
攻略

⬇️  
下载

💬  
评论

⚠️  
举报

# 客户机/服务器风格实例

- AirCG.3D三维网络字幕系统是一套对电视台字幕部分进行集中管理及播放的系统。

AirCG.3D演播室在线包装

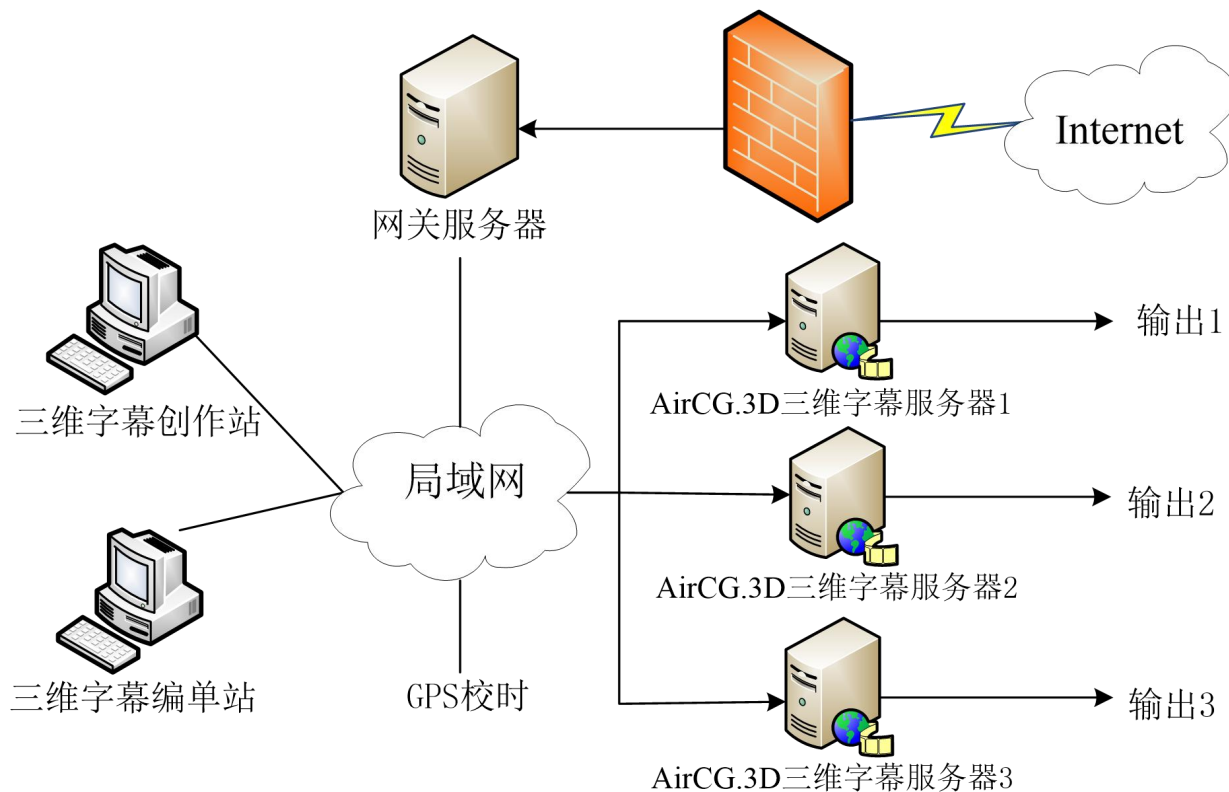




# 客户机/服务器风格实例

- 应用实例

- AirCG.3D三维网络字幕系统的结构



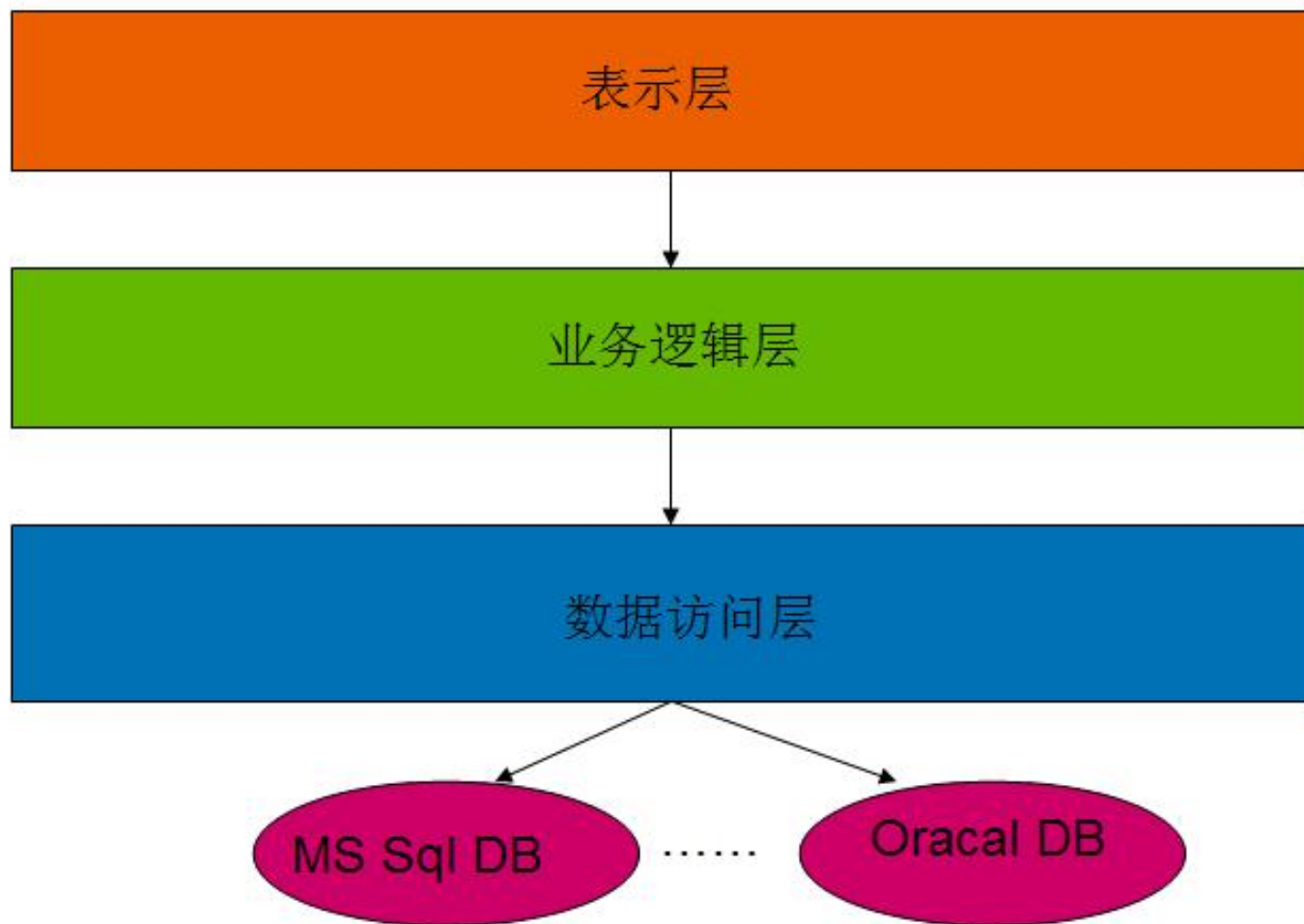
# 浏览器/服务器风格实例

- 应用实例：PetShop
- PetShop 是一个范例，微软用它来展示.Net企业系统开发的能力。
- PetShop是一个小型的项目，系统架构与代码都比

较简单，但凸现了许多颇有价值的设计与开发理念。

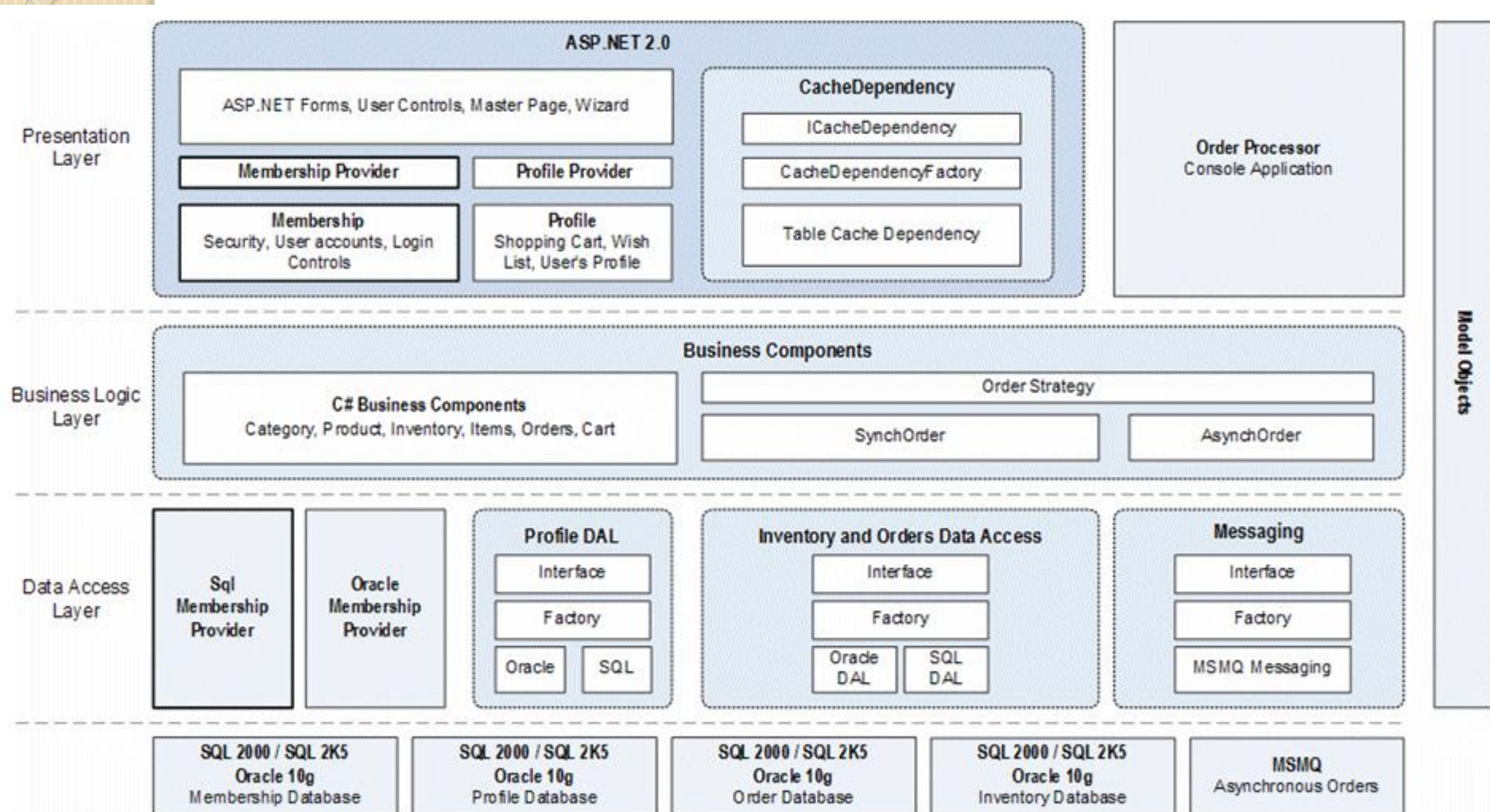


# PetShop的系统架构设计





# PetShop 4的系统架构设计



# 浏览器/服务器风格实例

PetShop架构分为三层：

(1) 数据访问层：其功能主要是负责数据库的访问。

(2) 业务逻辑层：是整个系统的核心，它与这个系统的业务（领域）有关。PetShop的业务逻辑层的相关设计，均和网上宠物店特有的逻辑相关。

(3) 表示层：是系统的UI部分，负责使用者与整个系统的交互。在这一层中，理想的状态是不应包括系统的业务逻辑。表示层中的逻辑代码，仅与界面元素有关。



## 4.3 典型的软件架构风格

- (2) 调用/返回风格总结
- 3种典型的调用/返回风格：
  - 主程序/子程序
  - 面向对象风格
  - 层次结构
- 风格变种
  - 客户机/服务器风格
  - 浏览器/服务器风格